TemporalDifference

Implementazione "from scratch" e analisi sperimentale
di algoritmi di Reinforcement Learning su CartPole, Acrobot e Pong

Federico Sabbadini
M.Sc. Computer Engineering — Università degli Studi di Brescia
Corso di Reinforcement Learning, A.A. 2025/26

15 luglio 2026

Sommario

Ho implementato da zero i quattro algoritmi tabellari richiesti (Q-Learning, SARSA, Monte Carlo e TD(0)) e un agente Deep Q-Network in PyTorch, applicandoli a tre ambienti Gymnasium: CartPole, Acrobot e Pong. Coerentemente con la consegna, l'obiettivo non è la performance assoluta ma la *comprensione* del comportamento degli algoritmi e l'analisi critica dei risultati; ogni risultato è mediato su 3 seed. Il lavoro comprende le analisi di sensibilità a ε, α e γ, il confronto SARSA vs Q-Learning e MC vs TD(0), tre casi di fallimento espliciti e l'ablazione del DQN (target network ed experience replay) con il bonus Double DQN. Due risultati su Acrobot *contraddicono* le ipotesi da cui ero partito: li ho mantenuti e analizzati invece di aggiustarli, perché sono il contributo più istruttivo del progetto.

---

1 Introduzione

1.1 Obiettivi

Il progetto implementa *da zero* i principali algoritmi di Reinforcement Learning e li confronta su tre ambienti di difficoltà crescente. Come chiede la consegna, l'enfasi è sulla **comprensione** e sull'**analisi sperimentale critica**, non sulla performance assoluta: diversi risultati sono volutamente sub-ottimali e vengono analizzati come tali, inclusi i casi di fallimento e i risultati che smentiscono le ipotesi da cui ero partito.

1.2 Ambienti e algoritmi

Ho lavorato su tre ambienti Gymnasium: **CartPole-v1** (comune; stato 4-D continuo, 2 azioni, reward +1/passo, denso), **Acrobot-v1** (assegnato; 6-D continuo, 3 azioni, reward -1/passo fino all'obiettivo, sparso) e **ALE/Pong-v5** (assegnato; Atari da pixel, tramite l'Arcade Learning Environment). La progressione non è casuale: CartPole serve a verificare che gli algoritmi siano corretti, Acrobot a metterli in crisi, Pong a mostrare cosa serve quando la tabella non è più praticabile.

Su CartPole e Acrobot uso quattro algoritmi tabellari scritti a mano: due di *controllo* (Q-Learning, SARSA) e due di *predizione* (Monte Carlo, TD(0)). Poiché entrambi gli ambienti hanno stato continuo, ho dovuto introdurre una **discretizzazione**: non è un dettaglio implementativo ma la scelta che, come mostro nella Sez. 5.6, governa il tetto di performance raggiungibile. Per Pong ho implementato un DQN in PyTorch (Mnih et al. 2015) con experience replay e target network, più il bonus Double DQN (van Hasselt et al. 2016).

MC e TD(0), essendo di predizione, valutano una policy *fissa* senza apprenderla: la policy da valutare deve quindi venire da qualche parte. Ho scelto di addestrare prima Q-Learning, estrarne la policy greedy e valutare *quella*, affiancandole una policy casuale come riferimento (Sez. 3.3). La scelta non è neutra: come si vedrà, il divario tra MC e TD(0) dipende proprio da quale policy si valuta.

2 Fondamenti teorici

Si riportano solo i concetti necessari a comprendere le implementazioni. La notazione segue Sutton & Barto (S&B nel seguito).

2.1 MDP, valore e policy

Un problema di RL è modellato come un *Markov Decision Process* (MDP): ad ogni *timestep* l'agente osserva uno *stato* s, sceglie un'*azione* a secondo una *policy* $\pi(a|s)$, riceve un *reward* r e transita in s'. L'obiettivo è massimizzare il *return* atteso $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, dove $\gamma \in [0, 1]$ è il fattore di sconto. Si definiscono la *state-value* $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ e la *action-value* $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$.

2.2 Predizione vs controllo

È fondamentale distinguere i due compiti, perché la consegna li richiede entrambi:

- **Predizione**: stimare V^π per una policy π *fissata*. Monte Carlo e TD(0) sono algoritmi di predizione.
- **Controllo**: trovare una policy migliore. Q-Learning e SARSA sono algoritmi di controllo.

2.3 Monte Carlo (MC) e TD(0)

Entrambi stimano V^π ma con target diversi (S&B §5.1, §6.1):

$$\text{MC:} \quad V(S_t) \leftarrow V(S_t) + \alpha \, [\, G_t - V(S_t) \,] \tag{1}$$

$$\text{TD(0):} \quad V(S_t) \leftarrow V(S_t) + \alpha \, [\, R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \,] \tag{2}$$

MC usa il return *effettivo* G_t: è non distorto (*unbiased*) ma ad alta varianza e richiede episodi completi. TD(0) usa il *bootstrap* $R_{t+1} + \gamma V(S_{t+1})$: è distorto (usa una stima) ma a varianza più bassa e aggiornabile online. Il termine $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ è il *TD error*.

L'origine del compromesso è intuitiva. Il return G_t di MC è la somma dei reward di un'*intera* traiettoria: ogni transizione casuale lungo l'episodio contribuisce alla sua variabilità, che si accumula tanto più l'episodio è lungo — per questo su episodi lunghi la stima MC oscilla molto più di quella su episodi corti. TD(0), invece, guarda un solo passo reale e poi si affida alla propria stima $V(S_{t+1})$: introduce poca varianza (una sola transizione casuale), ma finché V è ancora sbagliata quella stima trascina il target, da cui la distorsione. La distorsione si riassorbe man mano che V converge, ma la velocità con cui ciò accade dipende da quanto lo stato è "lontano" dal reward: è esattamente ciò che l'esperimento di predizione (Sez. 5.4) rende visibile.

2.4 SARSA (on-policy) e Q-Learning (off-policy)

Sono aggiornamenti TD sulla Q (S&B §6.4–6.5):

$$\text{SARSA:} \quad Q(S, A) \leftarrow Q(S, A) + \alpha \, [\, R + \gamma Q(S', A') - Q(S, A) \,] \tag{3}$$

$$\text{Q-Learning:} \quad Q(S, A) \leftarrow Q(S, A) + \alpha \, [\, R + \gamma \max_{a'} Q(S', a') - Q(S, A) \,] \tag{4}$$

SARSA è *on-policy*: il target usa l'azione A' effettivamente scelta dalla policy ε-greedy, quindi tiene conto del rischio dell'esplorazione. Q-Learning è *off-policy*: il target usa il max, cioè valuta la policy greedy indipendentemente da come si esplora.

La conseguenza pratica è che i due apprendono policy *qualitativamente* diverse quando l'esplorazione è rischiosa. L'esempio canonico è il CliffWalking (S&B Fig. 6.5): un percorso ottimo passa sul bordo di un dirupo, e una singola azione esplorativa lungo il bordo fa cadere l'agente. Q-Learning, il cui target assume comportamento greedy, impara il percorso ottimo *sul bordo* e paga care le esplorazioni; SARSA, che nel target include l'azione ε-greedy che davvero eseguirà, impara un percorso più interno e "prudente", che rende di più *finché si continua a esplorare*. Questa distinzione, in forma attenuata, spiega il confronto SARSA vs Q-Learning su CartPole (Sez. 5.1).

2.5 Esplorazione: ε-greedy

La policy ε-greedy sceglie un'azione casuale con probabilità ε (*exploration*) e l'azione greedy $\arg\max_a Q(s, a)$ altrimenti (*exploitation*): è il compromesso più semplice tra i due ed è usata in tutto il progetto (S&B §2.1). I metodi tabellari richiedono inoltre uno spazio di stati finito: la discretizzazione necessaria per CartPole e Acrobot, e le sue conseguenze, sono in Sez. 3.

2.6 Deep Q-Network (DQN)

Quando lo spazio degli stati è troppo grande (pixel di Pong), la Q viene approssimata da una rete neurale $Q(s, a; \theta)$ (Mnih et al. 2015). Due componenti stabilizzano l'addestramento:

- **Experience Replay**: le transizioni sono salvate in un buffer e campionate a mini-batch casuali, rompendo la correlazione temporale.
- **Target Network**: una copia θ^- della rete, aggiornata di rado, fornisce target stabili $y = R + \gamma \max_{a'} Q(S', a'; \theta^-)$.

Il **Double DQN** (van Hasselt et al. 2016) disaccoppia selezione e valutazione dell'azione, $y = R + \gamma\, Q(S', \arg\max_{a'} Q(S', a'; \theta); \theta^-)$, riducendo la sovrastima (*maximization bias*, S&B §6.7).

3 Metodologia e implementazione

3.1 Organizzazione del codice

Il codice sta nel pacchetto `rl_lab`: gli algoritmi sono funzioni pure (ambiente in ingresso, risultati in uscita), gli script orchestrano gli esperimenti, il plotting è isolato dai loop di training e gli iperparametri stanno in `configs.py`. Le componenti sono `algorithms/tabular/` (i quattro algoritmi), `algorithms/deep/` (rete Nature-CNN, replay buffer, loop DQN con i flag di ablazione e Double DQN), `envs/` (discretizzatore e preprocessing Atari), `experiments/runner.py` (esecuzione multi-seed: ogni esperimento passa di qui, quindi nessun risultato può essere accidentalmente a seed singolo) e `utils/` (seeding e plotting).

Tabella 1: Caratteristiche degli ambienti e discretizzazione adottata.

Ambiente	Stato	Azioni	Reward	Discretizzazione
CartPole	4-D cont.	2	+1/passo (denso)	10 bin/dim $\rightarrow 10^4$ stati
Acrobot	6-D cont.	3	−1/passo (sparso)	6 bin/dim $\rightarrow$ 46 656 stati
Pong	210×160 RGB	6	±1 (punto)	— (pixel, DQN)

3.2 Discretizzazione: le scelte che contano

Il wrapper `DiscretizedEnv` clippa ogni dimensione in un intervallo configurato, la scala e la tronca a un indice intero; il risultato è una tupla usata come chiave della Q-table (state aggregation, S&B §9.4). Due scelte incidono sui risultati e le dichiaro.

Intervalli di clipping. Su CartPole clippo posizione e angolo ai limiti di **terminazione** (±2.4 e ±0.21 rad), non a quelli dello spazio di osservazione (±4.8, ±0.418): oltre le soglie di terminazione l'episodio è già finito, quindi usare i limiti più larghi sprecherebbe metà dei bin di posizione e angolo su stati che non possono verificarsi. La scelta raddoppia la risoluzione utile sulle due dimensioni più importanti. Le velocità non hanno un limite ufficiale e le clippo a ±3.0, valore entro cui restano comodamente durante il bilanciamento. Su Acrobot i limiti *esistono* nel sorgente Gymnasium (4π e 9π per le due velocità angolari) e li riuso esattamente: nessun bin sprecato, nessuno saturo.

Numero di bin. È il parametro che non ho voluto assumere: la Sez. 5.6 lo misura direttamente, con esito opposto all'attesa.

3.3 Quale policy valutano MC e TD(0)?

Addestro Q-Learning, ne estraggo la policy greedy $\pi(s) = \arg\max_a Q(s, a)$ e valuto *quella* con entrambi gli stimatori; come riferimento valuto anche una policy *casuale*. Il doppio caso non è ridondante ed è anzi la parte più interessante del risultato (Sez. 5.4): la policy appresa genera episodi lunghi ($\approx$ 330 passi), quella casuale episodi corti ($\approx$ 22), e la lunghezza degli episodi è esattamente ciò che determina quanto TD(0) resta distorto.

Per confrontare gli stimatori traccio la stima $V(s_{\text{ref}})$ di un singolo stato, episodio per episodio. Come s_{ref} uso lo **stato di reset discretizzato**: è sempre raggiungibile ed è visitato all'inizio di ogni episodio, quindi entrambi gli stimatori lo aggiornano di continuo. La mia prima scelta — il "bin centrale" di ogni dimensione — era sbagliata: su Acrobot dà uno stato *fisicamente impossibile*, perché l'osservazione codifica gli angoli come $[\cos\theta, \sin\theta]$ e i due non possono essere entrambi ≈ 0. Quello stato non veniva mai visitato e la stima restava piatta a zero.

3.4 DQN, preprocessing Atari e riproducibilità

La rete è la Nature-CNN (Mnih et al. 2015): tre layer convoluzionali e due fully connected ($3136 \rightarrow 512 \rightarrow n_{\text{azioni}}$), che restituiscono $Q(s, \cdot)$ per tutte le azioni in un forward pass. I due stabilizzatori sono dietro un

flag ciascuno, così l'ablazione richiesta è un cambio di parametro e non di codice. Il replay buffer è limitato a 100k transizioni: il milione del paper Nature occuperebbe ≈ 28 GB, più della RAM disponibile.

Il preprocessing Atari segue lo standard di Mnih et al.: no-op iniziali casuali, frame-skip 4 con massimo pixel-wise sugli ultimi due frame (Atari fa lampeggiare gli sprite e la pallina può sparire a frame alterni), grayscale, resize 84×84 e stack di 4 frame — necessario perché da un solo fotogramma la direzione della pallina è ambigua. I frame restano `uint8` nel buffer e sono normalizzati solo al momento del training.

Ogni run fissa il seed con `set_seed` (numpy + torch), `env.reset(seed=...)` e `env.action_space.seed(...)` — quest'ultimo indispensabile, perché lo spazio delle azioni ha un RNG proprio da cui viene ogni azione esplorativa del DQN (vedi Sez. 7). Tutti i risultati sono media su 3 seed $\{42, 123, 456\}$. Gli array grezzi sono salvati in `results/` come `.npz` e **tutte le tabelle di questo report sono generate da quei file** (`report/make_report_tables.py`): nessun numero è trascritto a mano.

3.5 Origine del codice

I **quattro algoritmi tabellari** sono adattati dalle esercitazioni del corso (`control.py`, `prediction.py`): stessa regola di aggiornamento, riadattata all'API Gymnasium moderna e agli stati discretizzati. Il **DQN** (replay buffer, loop di training, target network, Double DQN), il **discretizzatore**, l'infrastruttura multi-seed e tutti gli script sono **scritti da zero**: il corso forniva solo un DQN basato su Stable-Baselines3, che la consegna vieta per il core. La Nature-CNN replica l'architettura di Mnih et al. (2015) con implementazione PyTorch originale. Nessuna riga di Stable-Baselines3 è usata nel progetto. Il dettaglio completo è in `docs/origine_codice.md`.

4 Setup sperimentale

4.1 Hardware, librerie, iperparametri

Gli esperimenti tabellari girano su CPU; il DQN su GPU NVIDIA RTX 5060 (8 GB, Blackwell), che richiede la build PyTorch CUDA 12.8. Librerie: Gymnasium (con ALE), PyTorch, NumPy, Matplotlib; dipendenze gestite con `uv` e bloccate in `uv.lock`. Nessuna libreria RL di alto livello è usata nel core, come richiesto.

Tabella 2: Iperparametri principali. I default tabellari seguono Sutton & Barto; quelli del DQN sono ridotti rispetto al paper Nature per rientrare nel budget di calcolo.

	Tabellari (CartPole / Acrobot)	DQN (Pong)
Budget	10 000 / 20 000 episodi	200 000 step ambiente
α	0.1	10^{-4} (Adam)
γ	0.99	0.99
ε	0.1 (costante)	$1.0 \rightarrow 0.02$ (lineare, 100k step)
Replay buffer	—	100 000 transizioni
Batch / target update	—	32 / ogni 1000 step
Learning starts	—	10 000 step
Seed	$\{42, 123, 456\}$ — 3 run per ogni esperimento	

Acrobot riceve il doppio degli episodi di CartPole perché il suo spazio degli stati è circa cinque volte più grande e il reward è sparso: con 10 000 episodi le curve non erano ancora entrate in regime.

Esperimenti. *CartPole*: Q-Learning vs SARSA; sensibilità a ε, α, γ per *entrambi* gli algoritmi (6 sweep); MC vs TD(0) su policy appresa e casuale; tre casi di fallimento; ε costante vs annealing. *Acrobot*: gli stessi confronti, più l'effetto della risoluzione di discretizzazione (4/6/8/10 bin). *Pong*: ablazione a cinque configurazioni (DQN completo, senza target network, senza replay, senza entrambi, Double DQN) più la baseline casuale.

4.2 Come vanno lette le figure

Le curve riportano il return per episodio, mediato sui 3 seed e lisciato con media mobile. Nei confronti a due curve (controllo, annealing, predizione, ablazione Pong) la banda ombreggiata è la deviazione standard $\pm 1\sigma$ tra i seed.

Nelle figure di **sensibilità** (quattro curve sovrapposte) ho invece *omesso deliberatamente le bande*: le regioni ombreggiate si coprirebbero a vicenda rendendo il grafico illeggibile. Ogni curva resta la media sugli stessi 3 seed, e la dispersione è riportata numericamente nelle tabelle. Lo dichiaro esplicitamente perché è una deviazione dalla lettera della consegna, fatta per leggibilità e non per nascondere varianza.

Le tabelle riportano la performance "finale" come media sull'**ultimo 10%** degli episodi di ciascun seed, aggregata poi tra seed: è una stima dell'asintoto molto più robusta del valore degli ultimissimi episodi, che sono rumorosi. Nelle curve Pong, dove seed diversi producono un numero diverso di episodi, la curva è disegnata solo fino al seed più corto, così ogni punto graficato è una media su 3 seed effettivi.

5 Risultati

Tutti i valori numerici provengono dagli array `.npz` salvati dai runner e sono calcolati da `make_report_tables.py`.

5.1 CartPole — controllo: Q-Learning vs SARSA

Entrambi gli algoritmi apprendono in modo netto (Fig. 1, Tab. 3): da $\approx$ 27–41 arrivano a 223.6 $\pm$ 49.4 (Q-Learning) e 258.3 $\pm$ 31.1 (SARSA). Due osservazioni contano più dei numeri assoluti.

Le curve sono ancora in salita a fine budget: a 10 000 episodi nessuna ha raggiunto un plateau, e il massimo dell'ambiente è 500. Il vincolo è il budget, non l'algoritmo: la griglia a 10^4 stati va esplorata quasi tutta.

SARSA supera Q-Learning ed è meno disperso tra seed ($\pm$31 contro $\pm$49) — l'opposto dell'intuizione, dato che Q-Learning apprende la policy *ottima*. La spiegazione: la curva misura la performance *durante* il training, con $\varepsilon = 0.1$ attivo. Q-Learning valuta la policy greedy e ignora che il 10% delle volte farà una mossa casuale; SARSA include quel rischio nel target e impara una policy che tollera le proprie deviazioni. Dove una singola azione sbagliata fa cadere l'asta, tenerne conto paga: è la dinamica del CliffWalking (S&B Fig. 6.5), attenuata ma visibile.

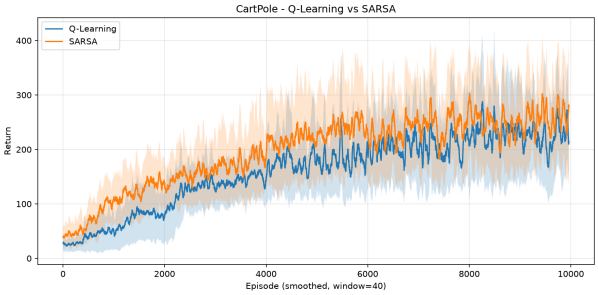


Figura 1: CartPole: Q-Learning vs SARSA a parità di iperparametri (media su 3 seed, banda $\pm 1\sigma$). SARSA (arancio) sta sopra per tutto il training; entrambe le curve salgono ancora a 10 000 episodi.

Tabella 3: CartPole, controllo: Q-Learning vs SARSA a parità di iperparametri ($\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.1$), media su 3 seed. Il return massimo dell'ambiente è 500; entrambe le curve sono ancora in salita a fine budget.

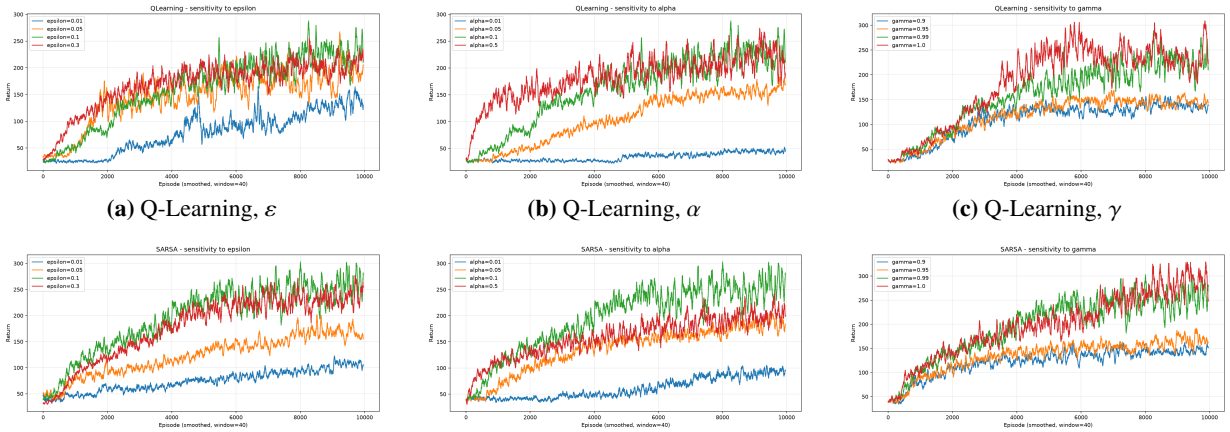
Algoritmo	Return iniziale (primi 100 ep)	Return finale (ultimo 10% ep, media $\pm$ std)	Episodi/seed
Q-Learning	26.6	223.6 $\pm$ 49.4	10000
SARSA	40.8	258.3 $\pm$ 31.1	10000

5.2 CartPole — sensibilità a ε, α, γ

La Fig. 2 riporta le sei sweep. Averle fatte per *entrambi* gli algoritmi non è ridondanza: su α i due si comportano diversamente, ed è il dato più interessante della sezione.

- γ **— l'effetto più netto.** Valori alti vincono per entrambi: $\gamma \in \{0.99, 1.0\}$ arriva a 230–280, $\gamma \in \{0.9, 0.95\}$ si blocca a 140–165. Strutturale: CartPole è un compito di *sopravvivenza* con reward +1/passo, quindi il valore di uno stato *è* quanto a lungo si sopravvive. Con $\gamma = 0.9$ l'orizzonte effettivo è $\approx 1/(1-\gamma) = 10$ passi: l'agente non può vedere la caduta che sta preparando.
- ε **— serve, ma il troppo non nuoce.** $\varepsilon = 0.01$ è il peggiore per entrambi (105–130), mentre $\varepsilon = 0.3$ *non* degrada (220–250, come 0.1): il reward denso perdona l'esplorazione, perché una mossa casuale costa al più qualche passo. Su Acrobot accadrà l'esatto contrario (Sez. 5.7): l'effetto di ε dipende dalla struttura del reward, non è una proprietà dell'algoritmo.

- α **— qui i due divergono.** Per Q-Learning $\alpha = 0.5$ e $\alpha = 0.1$ finiscono appaiati (200–220), con 0.5 più veloce all'inizio. Per *SARSA* invece $\alpha = 0.5$ parte forte ma **satura sotto** $\alpha = 0.1$ (≈ 210 contro ≈ 260). SARSA è on-policy, quindi il target $Q(S', A')$ contiene la casualità dell'azione A': un α alto insegue quel rumore invece di mediarlo. Q-Learning, che usa il max (deterministico dato Q), ne soffre meno. $\alpha = 0.01$ è troppo lento per entrambi (45–95).



(a) Q-Learning, ε **(b)** Q-Learning, α **(c)** Q-Learning, γ

(d) SARSA, ε **(e)** SARSA, α **(f)** SARSA, γ

Figura 2: CartPole: le sei sweep di sensibilità (media su 3 seed; bande omesse per leggibilità, Sez. 4.2). Riga superiore Q-Learning, inferiore SARSA. γ alto domina in entrambi; ε troppo basso è il vero problema, mentre $\varepsilon = 0.3$ non danneggia. La differenza tra i due algoritmi sta nel pannello α: per SARSA (e) $\alpha = 0.5$ (rosso) parte veloce ma satura *sotto* $\alpha = 0.1$ (verde), mentre per Q-Learning (b) i due si equivalgono.

5.3 CartPole — ε costante vs annealing

Mi aspettavo che l'annealing servisse a poco su CartPole, che ha reward denso e non è limitato dall'esplorazione. Il beneficio c'è, ma è soprattutto sulla *stabilità*: l'annealing ($1.0 \to 0.01$) chiude a 243.8 ± 23.9 contro 223.6 ± 49.4 del costante (Fig. 3). Il guadagno in media è modesto (+20), ma la dispersione tra seed si *dimezza*: ridurre ε a fine training toglie il rumore che la policy si porta dietro, e l'esito dipende meno dal seed. Il prezzo è un avvio più lento, fino a ≈ 3500 episodi.

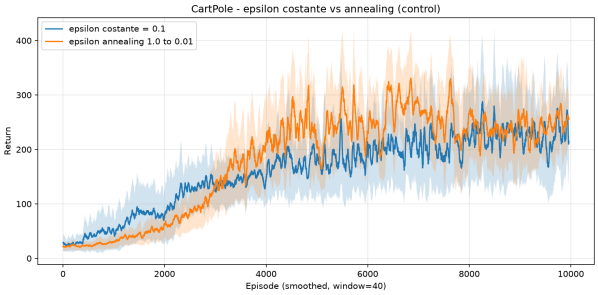


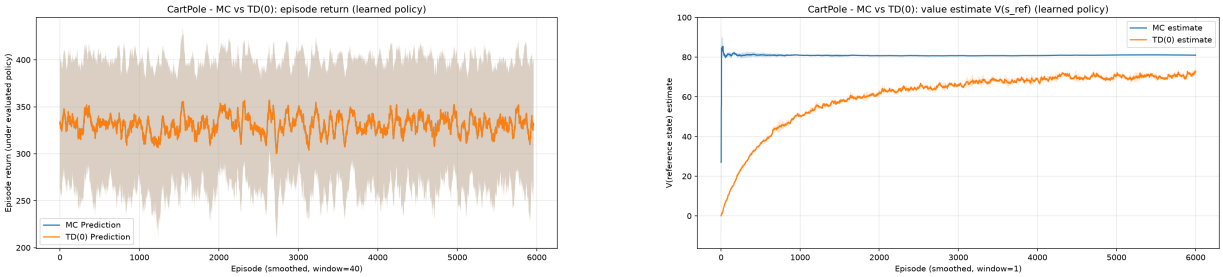
Figura 3: CartPole: ε costante = 0.1 vs annealing $1.0 \to 0.01$. L'annealing paga un avvio più lento, poi sorpassa e chiude con banda $\pm 1\sigma$ visibilmente più stretta.

5.4 CartPole — MC vs TD(0): bias, varianza, stabilità

Il return non può distinguere i due stimatori (Fig. 4a): le curve sono *identiche per costruzione*, perché MC e TD(0) valutano la *stessa* policy fissa e generano quindi gli stessi episodi (return ≈ 330). La curva di MC è letteralmente nascosta sotto quella di TD. Non è un errore di plotting: è la prova che la differenza tra i due metodi non vive nel return ma nelle stime $V(s)$. Per inciso, quei ≈ 330 passi sono ottenuti dalla policy greedy *senza* esplorazione, contro i ≈ 224 della curva di training: gran parte del divario dal massimo è il costo di ε, non incapacità della policy.

La differenza vive nelle stime (Fig. 4b), ed è il quadro classico di S&B §6.2. **MC** salta al valore corretto (81.07 ± 0.17) in poche decine di episodi ma con un picco iniziale violento: è *non distorto* e ad *alta varianza*, perché ogni aggiornamento usa il return completo di una traiettoria di ≈ 330 passi. **TD(0)** parte da 0 e sale lentamente e in modo liscio, fermandosi a 70.50 ± 0.28 dopo 6000 episodi — ancora 13% sotto MC: è *distorto* (fa bootstrap su una stima ancora sbagliata) ma a *bassa varianza*.

Il bias di TD(0) non è una costante: dipende dalla policy valutata. È il risultato che giustifica l'aver valutato anche la policy casuale (Fig. 5). Lì gli episodi durano ≈ 22 passi, lo stato di riferimento è vicinissimo alla terminazione, e TD(0) *raggiunge* MC entro ≈ 5000 episodi (17.04 contro 17.43): il bias sparisce. Con la policy appresa gli episodi sono quindici volte più lunghi e il bootstrap deve propagare il valore un passo alla volta. La distorsione di TD(0) è quindi una funzione della *distanza temporale* tra lo stato e il reward, non un difetto fisso dell'algoritmo.



(a) Return: identico per costruzione. **(b)** Stima $V(s_{\text{ref}})$: qui divergono.

Figura 4: CartPole, MC vs TD(0) sulla **policy appresa**. (a) Stessa policy, stessi episodi: il return non può dire nulla sugli stimatori. (b) Le stime invece raccontano tutto: MC (blu) non distorto e ad alta varianza iniziale, TD(0) (arancio) liscio ma distorto verso l'inizializzazione e non ancora convergente a 6000 episodi.

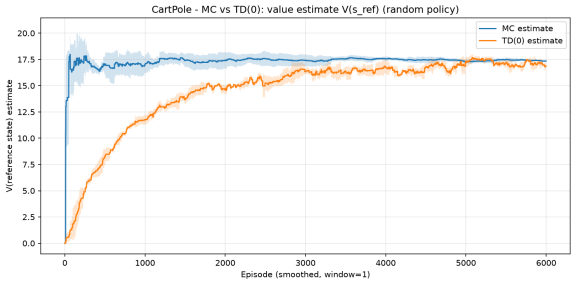


Figura 5: CartPole, MC vs TD(0) sulla **policy casuale**. Con episodi corti (≈ 22 passi) il bootstrap propaga in fretta e TD *raggiunge* MC: il bias dipende da quanto lo stato è lontano dalla terminazione. Da confrontare con la Fig. 4b.

Le figure del return sulla policy casuale e su Acrobot esistono in `figures/` ma non sono riportate: mostrano lo stesso fenomeno di Fig. 4a (curve sovrapposte per costruzione) e non aggiungono informazione.

5.5 Acrobot — controllo

Entrambi gli algoritmi migliorano da ≈ -470 (vicino al casuale, -500) fino a -341.4 ± 27.8 (Q-Learning) e -348.6 ± 3.2 (SARSA), restando lontanissimi dall'ottimo (≈ -100): l'agente impara qualche spinta utile, non uno swing-up efficiente (Fig. 6, Tab. 4).

Il dato più istruttivo è la **forma** della curva: il grosso del miglioramento avviene entro i primi ≈ 5000 episodi, poi le curve oscillano attorno a $-340/-350$ *senza migliorare* per altri 15 000. Non è lentezza, è un tetto: raddoppiare il budget non lo sposta. Con 46 656 stati e un reward informativo solo quando l'obiettivo viene raggiunto per caso, la Q-table a 6 bin non ha la risoluzione per rappresentare una politica migliore — ipotesi che la sezione seguente verifica.

Notevole l'inversione rispetto a CartPole sulla varianza: qui è **SARSA** ad essere drasticamente più stabile (±3.2 contro ±27.8). La stabilità relativa dei due algoritmi non è una loro proprietà intrinseca ma dipende dall'ambiente; la Sez. 7 riprende il punto.

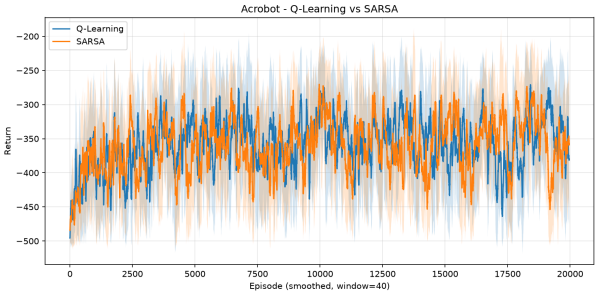


Figura 6: Acrobot: Q-Learning vs SARSA. Miglioramento reale rispetto al casuale (−500), ma plateau precoce (≈ 5000 episodi) e molto lontano dall'ottimo (≈ −100).

Tabella 4: Acrobot, controllo: Q-Learning vs SARSA, media su 3 seed. Policy casuale ≈ −500; un buon swing-up raggiunge ≈ −100.

Algoritmo	Return iniziale (primi 100 ep)	Return finale (ultimo 10% ep, media ± std)	Episodi/seed
Q-Learning	-468.0	−341.4 ± 27.8	20000
SARSA	-473.2	−348.6 ± 3.2	20000

5.6 Acrobot — risoluzione della discretizzazione

È l'esperimento che ha smentito la mia ipotesi di partenza, ed è il risultato più istruttivo del progetto. L'ipotesi: aumentare i bin *peggiora* l'apprendimento, perché lo spazio cresce come bin^6 e a 10 bin (10^6 stati) la tabella diventa irrimediabilmente sparsa — la curse of dimensionality da manuale.

I dati dicono l'**opposto**, in modo monotono e non ambiguo (Fig. 7): a 20 000 episodi **10 bin è la configurazione migliore** (≈ −280) e **4 bin la peggiore** (≈ −430), con 8 e 6 ordinatamente in mezzo.

Avevo confrontato il costo sbagliato. Le dinamiche di Acrobot richiedono *precisione angolare*: con 4 bin per dimensione, configurazioni del pendolo che richiedono coppie *opposte* finiscono nello *stesso* stato discreto. La Q-table non può rappresentare politiche diverse per stati che non sa distinguere, e nessun numero di episodi lo ripara — infatti la curva a 4 bin resta piatta e rumorosa fino in fondo. Questo costo di *aliasing* domina quello della sparsità: la curse of dimensionality è reale in principio, ma a questo budget non è ancora il vincolo attivo. È anche la giustificazione empirica del passaggio al DQN: per avere insieme precisione e generalizzazione serve un approssimatore di funzione.

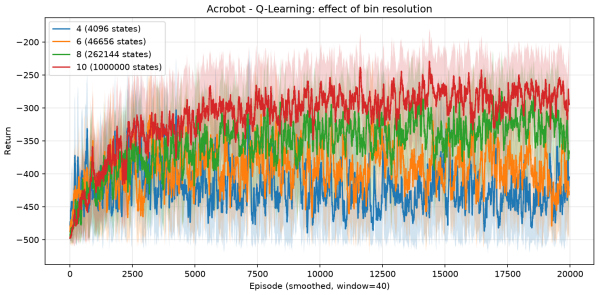


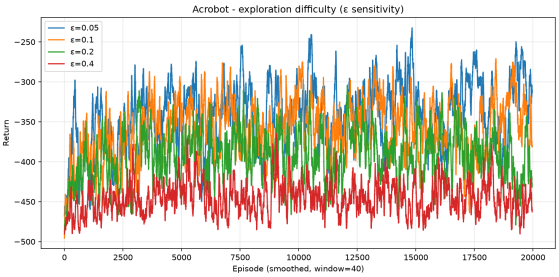
Figura 7: Acrobot: effetto della risoluzione di discretizzazione (Q-Learning, 20 000 episodi). Contrariamente all'ipotesi, 10 bin (10^6 stati, rosso) è la configurazione *migliore* e 4 bin (blu) la peggiore, con ordinamento monotono: 4 bin non riescono a distinguere stati che richiedono coppie opposte.

5.7 Acrobot — esplorazione: ε costante e annealing

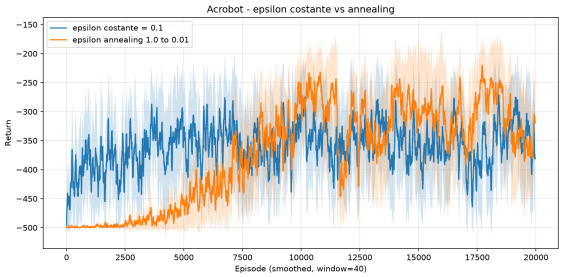
Anche qui l'attesa era chiara — reward sparso $\Rightarrow$ serve *più* esplorazione — e anche qui la sweep a ε costante dice il contrario (Fig. 8a): i valori *bassi* vincono ($\varepsilon = 0.05$ arriva a ≈ -310) e $\varepsilon = 0.4$ è nettamente il peggiore (≈ -450), di nuovo in ordine monotono.

Il punto non è che l'esplorazione sia inutile, ma che **l'esplorazione *costante* è lo strumento sbagliato**: con $\varepsilon = 0.4$ l'agente continua a iniettare coppie casuali *anche dopo* aver trovato uno swing-up, distruggendolo ad ogni episodio.

L'annealing conferma la lettura e chiude il cerchio (Fig. 8b): lo schedule $1.0 \to 0.01$ raggiunge -316.3 ± 24.5 contro -341.4 ± 27.8 del miglior ε costante — il guadagno più grande ottenuto su Acrobot senza toccare l'algoritmo. Il prezzo è visibile: per ≈ 5000 episodi la curva annealed è inchiodata a -500 (con $\varepsilon \approx 1$ l'agente è di fatto casuale), sorpassa attorno a 9000 e da lì resta sopra. Il confronto con CartPole, dove lo stesso schedule dà un guadagno modesto, isola il meccanismo: l'annealing paga dove l'esplorazione è davvero il collo di bottiglia.



(a) ε costante: basso batte alto.



(b) Annealing $1.0 \to 0.01$ vs costante 0.1.

Figura 8: Acrobot, esplorazione. (a) A ε *costante* i valori bassi vincono: un ε alto tenuto fisso continua a distruggere lo swing-up appreso. (b) Ma l'esplorazione *decrescente* è la scelta giusta: paga ≈ 5000 episodi a -500 e poi supera stabilmente il miglior ε costante. Insieme, i due pannelli dicono che il problema non era "quanta" esplorazione, ma "quando".

5.8 Acrobot — MC vs TD(0)

Il bias di TD(0) è qui ancora più marcato che su CartPole (Fig. 9), e conferma l'interpretazione data sopra. MC converge quasi subito a -91.17 ± 0.27; TD(0) parte da 0, *scende lentamente* e dopo 6000 episodi è solo a -62.00 ± 0.03. Lo stato di riferimento è a ≈ 90 passi dall'obiettivo e TD propaga il valore un passo per volta: per uno stato così distante dal reward il bootstrap è lentissimo e la stima resta ancorata all'inizializzazione.

Si noti la banda: quella di TD(0) è così stretta da risultare invisibile (± 0.03), quella di MC è ampia nei primi episodi. È il compromesso bias/varianza nella sua forma più pura: TD è molto *preciso* nel dare la risposta *sbagliata*, e ci mette molto ad arrivare a quella giusta.

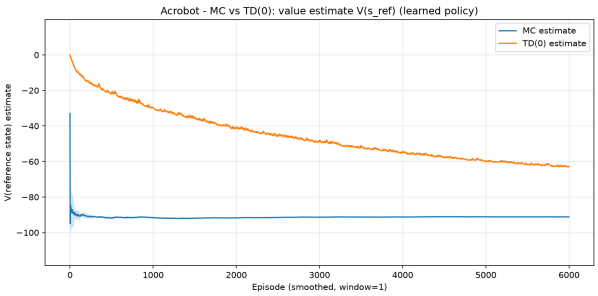


Figura 9: Acrobot, MC vs TD(0): stima $V(s_{\text{ref}})$ della policy appresa. MC (blu) converge subito a ≈ -91; TD(0) (arancio) scende da 0 con estrema regolarità ma dopo 6000 episodi è ancora a ≈ -62.

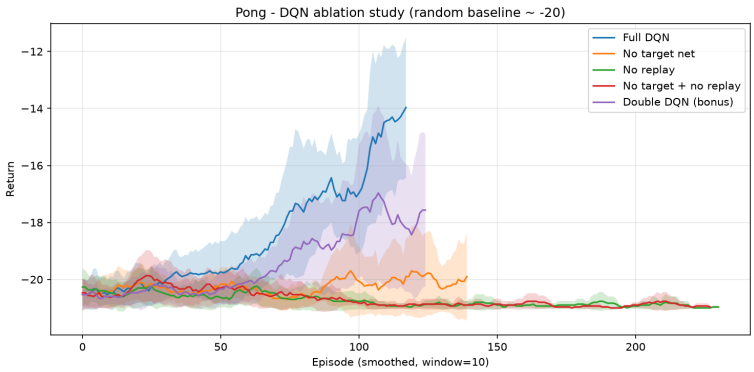


Figura 10: Pong: ablazione DQN (200k step, 3 seed, baseline casuale ≈ −20). Contiene entrambe le ablazioni richieste dalla consegna: *con vs senza replay* (blu vs verde) e *con vs senza target network* (blu vs arancio). Solo il DQN completo (blu) apprende su tutti i seed; il Double DQN (viola) apprende più lentamente; senza target network (arancio) la banda enorme segnala che la maggioranza dei seed non decolla; le due configurazioni senza replay (verde e rosso) restano piatte. Le figure per-ablazione isolate sono in `figures/pong/`.

5.9 Pong — ablazione del DQN

Rispetto alla baseline casuale (≈ −20.4) e con soli 200k step (Fig. 10, Tab. 5). Oltre alla media su 3 seed riporto tra parentesi *quanti seed su 3* hanno effettivamente imparato (return finale sopra −19): su soli 3 seed la deviazione standard è essa stessa molto rumorosa e può dare barre d'errore ingannevolmente strette, e il conteggio dei seed che apprendono distingue "la configurazione non impara" da "impara solo a volte" — una differenza che la sola media appiattisce.

- **DQN completo**: −13.5 ± 1.1 (3/3). Apprende in modo netto e su *tutti* i seed: il requisito della consegna ("mostrare apprendimento rispetto a policy casuale") è soddisfatto con ampio margine.
- **Double DQN** (bonus): −17.6 ± 2.3 (2/3). Apprende, ma più lentamente del completo e con un seed che non decolla.
- **Senza target network**: −20.0 ± 1.4 (1/3). È il risultato centrale: due seed su tre *restano al livello casuale*, uno solo impara.
- **Senza replay / senza entrambi**: −20.9 ± 0.0 e −20.9 ± 0.0 (0/3) — *nessun apprendimento*, curva piatta al livello casuale su ogni seed.

Tabella 5: Ablazione DQN su Pong: performance finale dopo 200k step ambiente, media ± std su 3 seed. Policy casuale ≈ −20.4.

Configurazione	Return finale (ultimo 10% ep, media ± std)	Episodi/seed (media)
DQN completo (replay + target)	−13.5 ± 1.1	137
Double DQN (bonus)	−17.6 ± 2.3	174
Senza target network	−20.0 ± 1.4	206
Senza experience replay	−20.9 ± 0.0	241
Senza target + senza replay	−20.9 ± 0.0	239

Entrambi i componenti sono quasi indispensabili, e questo è il punto (Fig. 10). Il *replay è necessario*: senza, la rete si addestra sul flusso di transizioni consecutive fortemente correlate e in 200k step non impara nulla su nessun seed — curva piatta. Ma anche la *target network* qui non è un semplice acceleratore: *senza di essa l'apprendimento diventa una lotteria*. Il valore medio (−20.0) nasconde il fatto reale, leggibile nella banda enorme e nel per-seed: un solo seed sale a ∼−18, gli altri due restano piatti a ∼−21. La causa è il "bersaglio mobile" portato all'estremo dal budget ridotto — ogni passo di gradiente sposta anche il target, quindi l'addestramento è in equilibrio instabile e il suo esito è deciso dalle condizioni iniziali che il seed

controlla (pesi della rete e prime esperienze esplorative): un seed prende trazione e apprende, gli altri due non decollano affatto. La lettura corretta non è quindi "replay necessario, target utile", ma: *il replay rende l'apprendimento possibile, la target network lo rende affidabile*; solo con entrambi il DQN impara su tutti e 3 i seed. Lo conferma un segnale indiretto: il DQN completo gioca *meno* episodi (137 contro ≈ 240), perché imparare allunga gli scambi e a parità di step le partite sono meno.

Il **Double DQN** non supera il DQN base, anzi lo peggiora leggermente (-17.6 contro -13.5, con un seed che collassa). Corregge la sovrastima dovuta al max su stime rumorose, che si accumula su orizzonti lunghi; a 200k step (lo 0.4% del budget Nature) quel danno non c'è ancora, mentre disaccoppiare selezione e valutazione aggiunge un po' di rumore. Il bonus è implementato e funzionante — in questo regime breve non c'è il male che dovrebbe curare.

6 Failure analysis

La consegna chiede almeno un caso di fallimento: ne riporto **tre**, ottenuti portando all'estremo un iperparametro alla volta su CartPole con Q-Learning (Fig. 11). Sono scelti perché ciascuno rompe un ingranaggio *diverso*: il passo di aggiornamento, l'esplorazione, l'assegnazione del credito. In tutti e tre il codice gira senza errori — ed è esattamente il punto: la correttezza dell'implementazione non garantisce l'apprendimento.

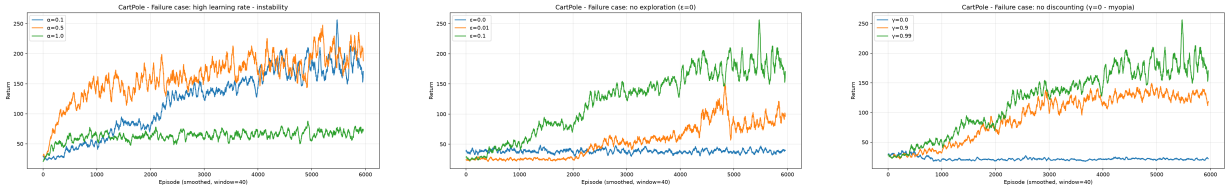
1. $\alpha = 1.0$ **— instabilità (mancata convergenza).** L'aggiornamento *sostituisce* completamente la stima ad ogni passo ($Q(S,A) \leftarrow R + \gamma \max_a Q(S',a)$): la media incrementale scompare, quindi Q non può mediare sulla stocasticità delle transizioni e insegue l'ultimo campione visto. L'agente si blocca a ≈ 65 per 6000 episodi.

La lettura importante è però un'altra: il fallimento è specifico del valore *estremo*, non dell'"α alto". Nella stessa figura $\alpha = 0.5$ non solo non fallisce, ma è la curva *migliore* (≈ 210, sopra $\alpha = 0.1$). Se avessi testato solo $\{0.1, 1.0\}$ avrei concluso, sbagliando, che "α grande destabilizza": la transizione non è graduale, c'è una soglia e sta tra 0.5 e 1.0.

2. $\varepsilon = 0$ **— comportamento subottimale.** L'agente è greedy dal primo istante su una Q-table inizializzata a zero: si aggrappa alla prima azione che sembra funzionare e non prova mai le alternative. La curva resta piatta a ≈ 40, mentre $\varepsilon = 0.1$ arriva a ≈ 170. Notevole il caso intermedio $\varepsilon = 0.01$: non fallisce del tutto, ma resta bloccato per ≈ 2500 episodi prima di sbloccarsi e arrivare a ≈ 100. L'esplorazione insufficiente non impedisce l'apprendimento, lo rende estremamente inefficiente (S&B §2.1).

3. $\gamma = 0$ **— miopia (fallimento del credit assignment).** È il più interessante dei tre, perché non è "lento" ma *strutturalmente impossibile*. Con $\gamma = 0$ il target si riduce al reward immediato; ma su CartPole il reward immediato è $+1$ ad *ogni* passo non terminale, *qualunque* azione si scelga. Tutte le coppie stato-azione collassano sullo stesso valore, la policy greedy non ha modo di preferire un'azione all'altra e degenera in una policy casuale: la curva resta piatta a ≈ 22, cioè *sotto* il punto di partenza degli altri.

Il caso mostra concretamente *a cosa serve* il discount factor: non a "pesare meno il futuro", ma a creare il canale attraverso cui l'informazione sulla caduta futura risale fino alle azioni che l'hanno causata. Senza $\gamma > 0$ quel canale non esiste, e nessun numero di episodi può rimediare.



(a) $\alpha = 1.0$: instabilità. **(b)** $\varepsilon = 0$: subottimale. **(c)** $\gamma = 0$: miopia.

Figura 11: CartPole, i tre casi di fallimento (Q-Learning, media su 3 seed). (a) $\alpha = 1.0$ (verde) si blocca a ≈ 65, ma $\alpha = 0.5$ (arancio) è la curva migliore: la soglia di instabilità sta tra i due, non "in alto". (b) $\varepsilon = 0$ (blu) resta piatto a ≈ 40; $\varepsilon = 0.01$ (arancio) apprende, ma con 2500 episodi di ritardo. (c) $\gamma = 0$ (blu) è piatto a ≈ 22: con reward $+1$ ovunque tutte le azioni valgono uguale e la policy greedy è indistinguibile da una casuale.

Un fallimento non provocato: il tetto di Acrobot. Ai tre fallimenti indotti va aggiunto quello *spontaneo* di Sez. 5.5: su Acrobot le curve si fermano a ≈ -345 e non migliorano per 15 000 episodi. Non è un bug né un iperparametro sbagliato, è il limite rappresentativo della Q-table discretizzata — e la Sez. 5.6 lo dimostra facendo variare l'unica cosa che lo sposta davvero: il numero di bin.

7 Analisi critica

Questa sezione raccoglie le letture *trasversali*, che nessun singolo esperimento mostra da solo.

7.1 SARSA vs Q-Learning: la teoria non predice il vincitore

Il confronto ha dato esiti *opposti* nei due ambienti, ed è istruttivo proprio per questo. Su **CartPole** SARSA vince su entrambi i fronti (return 258 vs 224, std ±31 vs ±49). Su **Acrobot** Q-Learning ottiene un return leggermente migliore (−341 vs −349) ma SARSA è quasi dieci volte più stabile tra i seed (±3.2 vs ±27.8).

La differenza teorica è sempre la stessa — il target di Q-Learning usa il max e ignora l'esplorazione, quello di SARSA usa l'azione realmente scelta e la include — ma le *conseguenze* dipendono dall'ambiente. Dove l'esplorazione è costosa (CartPole: una mossa casuale può far cadere l'asta), SARSA guadagna perché apprende una policy robusta al proprio rumore. Dove il reward è sparso (Acrobot), il max amplifica le rare traiettorie fortunate che raggiungono l'obiettivo: quali siano dipende dal seed, da cui i picchi migliori ma la varianza altissima di Q-Learning. Concluderne che "off-policy è meglio" non ha valore predittivo: conta come il rischio dell'esplorazione interagisce con la struttura del reward.

7.2 Perché le curve non convergono mai davvero

Nessuna curva tabellare si appiattisce del tutto, e non deve: con ε e α *costanti* l'agente continua a esplorare e pesa le esperienze recenti in modo esponenziale (media a pesi mobili, non media esatta), quindi la fluttuazione asintotica è attesa (S&B §2.5). Che l'annealing di ε riduca sensibilmente la varianza tra seed su entrambi gli ambienti è la conferma sperimentale di questa lettura.

7.3 Due ipotesi smentite, e perché le ho tenute

I risultati sui bin e su ε in Acrobot contraddicono entrambi la mia previsione iniziale. Non ho ritoccato né gli esperimenti né le ipotesi: sono riproducibili sui 3 seed, *monotoni* nel parametro (quindi non sono rumore) e hanno una spiegazione meccanicistica coerente.

Il punto interessante è che le due "regole" che ho smentito sono vere *in astratto*: la curse of dimensionality esiste, e un reward sparso richiede davvero più esplorazione. Sono però affermazioni *asintotiche*, mentre un esperimento avviene a budget finito e su un ambiente specifico — il tipo di errore che solo la verifica sperimentale intercetta.

7.4 Limiti metodologici che riconosco

- **Predizione: misura locale.** Il confronto MC vs TD(0) traccia $V(s_{\text{ref}})$ di *un* singolo stato; una misura globale (RMSE di V su tutti gli stati, come nel *random walk* del corso) richiederebbe un V di riferimento che qui non ho. L'attenuante è che valutare *due* policy diverse mi ha comunque permesso di isolare *da cosa* dipende il bias.
- **Asse x in "episodi" nell'ablazione Pong.** A parità di 200k step le configurazioni producono numeri diversi di episodi, quindi confrontarle allo stesso indice non è un confronto a parità esatta di calcolo. Un asse in step ambiente sarebbe più corretto; la conclusione qualitativa non cambia, dato che le curve senza replay restano piatte per l'intera durata.
- **Persistenza parziale dei dati grezzi.** Controllo, annealing, predizione e ablazione Pong sono salvati in `.npz`; le sweep di sensibilità e i fallimenti producono solo figure, quindi le tabelle usano solo dati persistiti.
- **Bande assenti nelle figure di sensibilità**, per leggibilità (Sez. 4.2).
- **Il DQN è valutato durante il training**, con $\varepsilon > 0$: è una stima leggermente *pessimistica* della policy greedy. I checkpoint sono salvati, quindi una valutazione greedy separata sarebbe fattibile senza riaddestrare.

7.5 Problemi implementativi incontrati

Distinti dai limiti intrinseci dell'RL, questi sono ostacoli reali che ho dovuto risolvere:

- **GPU Blackwell (RTX 5060)**: richiede la build PyTorch CUDA 12.8; con build precedenti la GPU non viene rilevata e il training cade silenziosamente su CPU.
- **Pipeline Atari, tre trappole**: (i) l'ambiente `-v5` ha già un frame-skip interno che, combinato con `AtariPreprocessing`, darebbe uno skip effettivo di 16 frame, quindi va forzato `frameskip=1`; (ii) in Gymnasium 1.x `FrameStack` si chiama `FrameStackObservation`; (iii) i frame impilati sono già channel-first `(4,84,84)` e non vanno trasposti. Sbagliare uno dei tre produce un crash oppure — peggio — input silenziosamente malformati.
- **Memoria del replay buffer**: 1M di transizioni Atari sarebbero ≈ 28 GB, quindi ho limitato il buffer a 100k tenendo i frame in `uint8`.
- **DQN non riproducibile per un RNG nascosto.** Run che avrebbero dovuto essere identiche divergevano: l'azione esplorativa viene da `env.action_space.sample()`, e lo spazio delle azioni ha un *RNG separato* che né `set_seed` né `reset(seed=)` inizializzano. All'inizio ($\varepsilon \approx 1$) quasi tutta l'esperienza nasceva da quel RNG non seminato. Aggiunto `env.action_space.seed(seed)`; i risultati Pong riportati sono di questo run corretto e riproducibile.
- **Episodio troncato contato come completo.** Quando il budget di step finiva a metà partita, il return parziale (troppo ottimistico) veniva registrato come episodio, con uno spurio picco verso l'alto sul bordo destro di ogni curva. Corretto scartando l'episodio non terminato.
- **Stato di riferimento impossibile** (Sez. 3.3): il grafico piatto a zero era corretto, era la domanda ad essere mal posta.

Nel complesso i risultati mostrano apprendimento, confronti corretti e fallimenti analizzati, e in questo sono adeguati agli obiettivi della consegna; non sono però "research-grade", e i divari li dichiaro apertamente nelle conclusioni.

8 Conclusioni

Rispondo esplicitamente alle tre domande richieste dalla consegna.

8.1 Cosa non ha funzionato

- **Acrobot non decolla**: plateau a ≈ -345 contro un ottimo di ≈ -100, e nessun miglioramento per 15 000 episodi. Raddoppiare il budget non serve, perché il limite è la risoluzione della Q-table (Sez. 5.5).
- **CartPole non raggiunge 500**: con 10^4 stati il tetto in 10 000 episodi è circa metà del massimo. Anche qui il vincolo è rappresentativo, non algoritmico.
- **I tre fallimenti indotti** (Sez. 6): $\alpha = 1.0$ non converge, $\varepsilon = 0$ resta subottimale, $\gamma = 0$ non può imparare affatto — e in tutti e tre il codice gira correttamente.
- **Double DQN non supera il DQN base** a 200k step, anzi lo peggiora: il maximization bias che dovrebbe correggere non ha ancora avuto il tempo di accumularsi (Sez. 5.9).
- **Il DQN non era riproducibile**: lo spazio delle azioni ha un RNG separato che né `set_seed` né `reset(seed=)` toccano, e da lì viene ogni azione esplorativa; due run con lo stesso seed divergevano. Scoperto e corretto (Sez. 7).
- **Il mio primo confronto MC vs TD(0) non misurava nulla**: graficava il return della policy condivisa, identico per costruzione tra i due stimatori — "giusto" e inutile (Sez. 5.4).

8.2 Cosa mi ha sorpreso

- **Le due ipotesi smentite su Acrobot**: più bin sono *meglio* (non peggio — l'aliasing di una griglia grossolana domina la curse of dimensionality), e *meno* esplorazione è meglio a ε costante, salvo poi essere battuta dall'annealing: il problema non era "quanta" esplorazione ma "quando" (Sez. 5.6, 5.7).
- **SARSA batte Q-Learning su CartPole**, con varianza minore. Mi aspettavo il contrario; ma la curva misura la performance *con l'esplorazione attiva*, e SARSA è l'unico dei due che ne tiene conto.
- **Il bias di TD(0) non è una costante**: sulla policy casuale (episodi corti) TD raggiunge MC, su quella appresa ($\approx$ 330 passi) resta indietro del 13% — dipende dalla distanza temporale dal reward, che il *random walk* del corso nasconde.
- **Senza target network l'apprendimento è una lotteria**: mi aspettavo un agente "più lento", ho trovato invece che 2 seed su 3 non decollano affatto e solo uno impara. La media (-20) nasconde il fenomeno, il per-seed lo rivela: la target network non accelera soltanto, rende l'apprendimento *affidabile* (Sez. 5.9).

8.3 Limiti degli algoritmi

- **Discretizzazione** — il collo di bottiglia dominante, e ha due lame: pochi bin non distinguono stati che richiedono azioni diverse, troppi bin fanno esplodere lo spazio. Nessuna scelta di bin risolve entrambi i problemi: serve l'approssimazione di funzione. È la motivazione empirica del DQN.
- **Efficienza campionaria e convergenza** — migliaia di episodi per risultati modesti (nessuna generalizzazione tra stati simili), e con α ed ε costanti la stima fluttua senza mai convergere davvero.
- **Sparsità del reward** — su Acrobot il segnale (-1 per passo) non dice *quale* azione fosse sbagliata: il credit assignment su 500 passi è il problema vero, e nessuno dei quattro algoritmi tabellari lo affronta.
- **Sensibilità agli iperparametri** — marcata, e in modi che cambiano da ambiente ad ambiente e persino da algoritmo ad algoritmo (α alto danneggia SARSA ma non Q-Learning).
- **Costo del Deep RL** — il DQN risolve la rappresentazione, ma 200k step sono lo 0.4% del budget Nature: è per questo che Pong si ferma a ≈ -13, e che senza target network metà dei seed non decolla.